

BỘ GIÁO DỤC VÀ ĐÀO TẠO
TRƯỜNG ĐẠI HỌC XÂY DỰNG HÀ NỘI
KHOA CÔNG NGHỆ THÔNG TIN



BÁO CÁO NHẬP MÔN TRÍ TUỆ NHÂN TẠO
ĐỀ TÀI 2: THUẬT TOÁN A*

Giảng viên bộ môn : Phạm Hồng Phong
Lớp học phần : 68CS3
Thành viên : Đỗ Công Trí - 0214268
Đào Đăng Quang- 0212168
Phạm Văn Thắng - 0213168
Nguyễn Tuấn Anh - 0203868
Nguyễn Đức Mạnh - 0210668

Hà Nội, 2025

Bảng Phân Công Công Việc

Người phụ trách	Nhiệm vụ
Đỗ Công Trí	Giới thiệu bài toán 8 ô
Phạm Văn Thắng	Tìm hiểu về thuật toán A*
Đào Đăng Quang	Cài đặt thuật toán A* cho bài toán 8 ô
Nguyễn Tuấn Anh	Mã nguồn
Nguyễn Đức Mạnh	Kết luận

Mục Lục:

I. GIỚI THIỆU BÀI TOÁN 8-PUZZLE	2
1. Giới thiệu tổng quan:	2
2. Một số ứng dụng của bài toán 8-puzzle	2
II. Thuật toán A*	3
1. Giới thiệu tổng quát:	3
2. Nguyên lý hoạt động:	3
3. Cài đặt thuật toán:	4
III. Cài đặt thuật toán A* cho bài toán 8-puzzle	5
IV. Mã nguồn	9
1. Khai báo ban đầu	9
2. Hàm tiện ích	9
3. Thuật toán A*	11
4. Chạy thử	13
5. Kết quả	14

Lời Mở Đầu

Trong lĩnh vực khoa học máy tính, giải thuật tìm kiếm là một thuật toán nhận đầu vào là một bài toán và trả về kết quả là lời giải cho bài toán đó, thường qua việc xem xét nhiều khả năng khác nhau. Tập hợp tất cả các khả năng có thể có của bài toán được gọi là không gian tìm kiếm. Các thuật toán tìm kiếm có thể được phân loại thành hai nhóm chính: thuật toán tìm kiếm không có thông tin (hay còn gọi là tìm kiếm sơ đẳng) và thuật toán tìm kiếm có thông tin. Các thuật toán tìm kiếm không có thông tin thường đơn giản và trực quan, trong khi các thuật toán tìm kiếm có thông tin sử dụng hàm đánh giá heuristic, giúp giảm đáng kể thời gian tìm kiếm và tối ưu hóa quá trình tìm kiếm lời giải.

Để áp dụng các giải thuật tìm kiếm vào thực tiễn, việc chuyển không gian tìm kiếm thành một đồ thị là vô cùng quan trọng. Đồ thị giúp ta mô tả một cách rõ ràng và ngắn gọn các mối liên hệ giữa các trạng thái trong bài toán, từ đó thuận tiện cho việc giải quyết. Trong bài báo cáo này, nhóm chúng em xin trình bày 3 thuật toán tìm kiếm cơ bản và phổ biến, dựa trên lý thuyết đồ thị là thuật toán : tìm kiếm A^* , tìm kiếm BFS(theo chiều rộng), tìm kiếm DFS(theo chiều sâu). Đặc biệt, chúng em sẽ tập trung áp dụng thuật toán tìm kiếm A^* để giải quyết bài toán 8 puzzle , một bài toán phổ biến trong giới lập trình. Chúng em sẽ phân tích cơ chế hoạt động của thuật toán, nêu ra ưu nhược điểm của từng giải thuật, cũng như đánh giá độ phức tạp của các thuật toán này.

I. GIỚI THIỆU BÀI TOÁN 8-PUZZLE

1. Giới thiệu tổng quan:

Bài toán 8-Puzzle là một trò chơi giải đố trí tuệ cổ điển và phổ biến. Đây là một trò chơi đơn giản nhưng đầy thách thức, thường được sử dụng để kiểm tra và phát triển các thuật toán tìm kiếm và trí tuệ nhân tạo. Dưới đây là một giới thiệu tổng quan về trò chơi này: Trò chơi 8 quân cờ thường chơi trên một bảng vuông 3x3 với 8 ô và một ô trống. Mục tiêu của trò chơi là sắp xếp các ô số từ 1 đến 8 theo một thứ tự cụ thể hoặc theo một trạng thái mà người chơi muốn di chuyển mỗi ô số hoặc ô trống sang một ô trống khác, nhưng chỉ có thể di chuyển 1 bước một lần. Di chuyển có thể thực hiện theo hướng lên, xuống, trái, hoặc phải. Mục tiêu của người chơi là di chuyển các ô số để đạt được trạng thái cuối cùng, thường là một sắp xếp theo thứ tự tăng dần từ trái sang phải và từ trên xuống dưới. Để giải quyết bài toán, ta có thể áp dụng các thuật toán tìm kiếm như A*, Tìm kiếm theo chiều sâu (DFS), tìm kiếm theo chiều rộng (BFS) để đánh giá và lựa chọn các nước đi chuyển.

1	2	3
4	5	6
7	8	

2. Một số ứng dụng của bài toán 8-puzzle

Bài toán 8-puzzle, mặc dù có vẻ đơn giản, nhưng có nhiều ứng dụng quan trọng trong các lĩnh vực khác nhau, đặc biệt là trong trí tuệ nhân tạo và khoa học máy tính. Dưới đây là một số ứng dụng chính của bài toán 8-puzzle:

- o Nghiên cứu Thuật toán Tìm kiếm và Giải quyết Vấn đề: bài toán 8-puzzle thường được sử dụng để minh họa cho các thuật toán tìm kiếm như tìm kiếm theo chiều rộng (BFS), tìm kiếm theo chiều sâu (DFS), A*, và các thuật toán khác.
- o Robot và tự động hóa: nguyên lý của bài toán 8-puzzle có thể áp dụng trong việc lập kế hoạch di chuyển cho robot. Các thuật toán giải bài toán 8-puzzle cũng có thể giúp tối ưu hóa đường đi của robot.
- o Phát triển trò chơi và giải trí: bài toán 8-puzzle và các biến thể của nó như là 15-puzzle thường được sử dụng trong các trò chơi giải đố và logic để thử thách người chơi. Thông qua 8-puzzle người chơi có thể rèn luyện tư duy logic và khả năng

giải quyết vấn đề của mình Bài toán 8-puzzle không chỉ là một bài toán học thuật mà còn có nhiều ứng dụng thực tiễn trong nhiều lĩnh vực khác nhau.

Việc nghiên cứu và giải quyết bài toán này giúp phát triển các kỹ năng và công nghệ mới, áp dụng vào các bài toán phức tạp hơn trong thực tế

Ta có thể dùng các thuật toán tìm kiếm như BFS, DFS, A*,... để giải quyết bài toán tìm kiếm trạng thái của puzzle trên. Nhưng các thuật toán tìm kiếm không có thông tin như BFS, DFS,... để đi từ trạng thái ban đầu đến trạng thái đích sẽ rất lâu và cần rất nhiều bước di chuyển để đạt được vì 2 thuật toán này sẽ duyệt từng trạng thái được sinh ra từ trạng thái ban đầu của nó. Vì vậy, các thuật toán tìm kiếm có thông tin như Greedy, A*,... sẽ giúp ta giải quyết bài toán này nhanh hơn và ít bước di chuyển hơn so với các thuật toán tìm kiếm không có thông tin. Nhóm em đã chọn thuật toán tìm kiếm có thông tin A* dựa theo hàm đánh giá Heuristic để đánh giá và tìm ra trạng thái tối ưu nhất trong các trạng thái được mở rộng nên tối ưu được các bước di chuyển cũng như tiết kiệm thời gian giải quyết cho bài toán tìm trạng thái đích của trò chơi ô chữ 8 số này.

II. Thuật toán A*

1. Giới thiệu tổng quát:

Trong khoa học máy tính, thuật toán A* (A-star) là một phương pháp tìm kiếm thông minh được sử dụng trong trí tuệ nhân tạo, đặc biệt trong các bài toán tìm đường đi ngắn nhất giữa hai điểm trong không gian đồ thị. Thuật toán A* kết hợp một cách hiệu quả hai yếu tố quan trọng: Chi phí thực tế từ điểm bắt đầu đến điểm hiện tại và dự đoán chi phí còn lại từ điểm hiện tại đến đích. Chính sự kết hợp này giúp A* tìm ra lộ trình tối ưu với tốc độ nhanh chóng và độ chính xác cao. Thuật toán A* được mô tả lần đầu vào năm 1968 bởi Peter Hart, Nils Nilsson, và Bertram Raphael. Trong bài báo của họ, thuật toán được gọi là thuật toán A; khi sử dụng thuật toán này với một đánh giá heuristic thích hợp sẽ thu được hoạt động tối ưu, do đó mà có tên A*.

Cấu trúc cơ bản của thuật toán A* bao gồm ba thành phần quan trọng:

- $g(n)$: Chi phí thực tế từ điểm bắt đầu đến điểm n . Đây là chi phí đã được tính toán dựa trên các bước di chuyển đã thực hiện, phản ánh mức độ tốn kém của quá trình di chuyển cho đến thời điểm hiện tại.
- $h(n)$: Ước tính chi phí từ điểm n đến đích, hay còn gọi là hàm heuristic. Hàm này giúp thuật toán đưa ra dự đoán về chi phí còn lại, từ đó xác định con đường nào sẽ có khả năng đưa ta đến đích một cách hiệu quả nhất.

- $f(n) = g(n) + h(n)$: Tổng chi phí tại điểm n , bao gồm chi phí thực tế ($g(n)$) và chi phí ước tính đến đích ($h(n)$). Hàm $f(n)$ đóng vai trò quyết định trong việc lựa chọn điểm tiếp theo để di chuyển, ưu tiên các điểm có tổng chi phí thấp nhất, nhằm đảm bảo lộ trình tìm kiếm nhanh chóng và tối ưu.

Nhờ vào việc kết hợp cả chi phí thực tế và dự đoán, thuật toán A^* có khả năng tìm ra lộ trình tối ưu trong hầu hết các bài toán tìm kiếm đường đi, đặc biệt là trong các môi trường phức tạp với nhiều lựa chọn.

2. Nguyên lý hoạt động:

Thuật toán A^* hoạt động thông qua một quy trình lặp lại, liên tục mở rộng các nút trong đồ thị cho đến khi tìm ra được đường đi tối ưu. Quy trình này có thể được chia thành các bước chính như sau:

Khởi tạo:

Thuật toán A^* bắt đầu bằng việc đưa điểm bắt đầu vào danh sách mở (open list). Mỗi điểm trong danh sách này sẽ chứa các thông tin về chi phí $g(n)$, hàm heuristic $h(n)$ và tổng chi phí $f(n)$.

Lặp lại tìm kiếm:

Mỗi vòng lặp, thuật toán A^* sẽ chọn điểm có giá trị $f(n)$ nhỏ nhất trong danh sách mở. Điểm này được gọi là nút hiện tại và thuật toán sẽ kiểm tra tất cả các điểm kế tiếp có thể di chuyển đến từ nút này. Sau khi kiểm tra, các điểm này sẽ được đưa vào:

- Danh sách đóng (closed list): Nếu điểm đó đã được kiểm tra trước, không cần kiểm tra lại.
- Danh sách mở (open list): Nếu điểm đó chưa được kiểm tra, sẽ được thêm vào danh sách mở để tiếp tục xử lý sau này.

Tính toán $f(n)$:

Khi thuật toán lựa chọn một điểm trong danh sách mở, nó sẽ tính toán giá trị $f(n)$ cho các điểm kế tiếp (nút con) dựa trên công thức $f(n) = g(n) + h(n)$. Sau đó, giá trị này sẽ được so sánh với giá trị $f(n)$ của điểm đã được kiểm tra trước đó.

Nếu đường đi mới đến một điểm có chi phí tổng $f(n)$ thấp hơn so với đường đi cũ, thuật toán sẽ cập nhật lại thông tin và lựa chọn đường đi mới này. Điều này giúp thuật toán tìm ra đường đi tối ưu và tối thiểu hóa chi phí.

Kết thúc:

Quá trình tìm kiếm tiếp tục cho đến khi một trong hai điều kiện sau xảy ra:

- Thuật toán tìm thấy điểm đích và đường đi tối ưu được xác định.
- Danh sách mở trống, tức là không còn điểm nào khả thi để kiểm tra và không có đường đi nào dẫn đến đích.

Điều khiển thuật toán A* trở nên hiệu quả và mạnh mẽ chính là việc sử dụng hàm heuristic $h(n)$. Nếu hàm này được thiết kế hợp lý, thuật toán A* sẽ có thể tìm ra đường đi tối ưu trong thời gian rất ngắn. Đặc biệt, nếu hàm heuristic là “thực tế (admissible)”, tức là không bao giờ ước lượng chi phí quá cao so với giá trị thực tế, thuật toán A* sẽ luôn tìm ra được giải pháp tối ưu.

3. Cài đặt thuật toán:

- 1.Open: tập các trạng thái đã được sinh ra nhưng chưa được xét đến.
- 2.Close: tập các trạng thái đã được xét đến.
- 3.Cost(p,q): là khoảng cách giữa p và q.
- 4.g(p): khoảng cách từ trạng thái đầu đến trạng thái hiện tại p.
- 5.h(p): giá trị được lượng giá từ trạng thái hiện tại đến trạng thái đích.
- 6.f(p) = g(p) + h(p)

□ Bước 1:

- o Open = {s}
- o Close = {}

□ Bước 2: while (Open != {})

□ Chọn trạng thái
(đỉnh) tốt nhất p
trong Open (xóa p
khỏi Open).

- ▪ Nếu p là trạng thái kết thúc thì thoát.
- ▪ Chuyển p qua Close và tạo ra các trạng thái kế tiếp q sau p .
- ▪ Nếu q đã có trong Open
- ▪ Nếu $g(q) > g(p) + \text{Cost}(p, q)$
- ▪ $g(q) = g(p) + \text{Cost}(p, q)$

- ▪ $f(q) = g(q) + h(q)$
- ▪ $\text{prev}(q) = p$
(đỉnh cha của q là p)
- Chọn trạng thái (đỉnh) tốt nhất p trong Open (xóa p khỏi Open).
- ▪ Nếu p là trạng thái kết thúc thì thoát.
- ▪ Chuyển p qua Close và tạo ra

các trạng thái kế
tiếp q sau p .

- ▪ Nếu q đã có
trong Open
- ▪ Nếu $g(q) >$
 $g(p) + \text{Cost}(p, q)$
- ▪ $g(q) = g(p) +$
 $\text{Cost}(p, q)$
- ▪ $f(q) = g(q) +$
 $h(q)$
- ▪ $\text{prev}(q) = p$
(đỉnh cha của q là
 p)

- Chọn trạng thái (đỉnh) tốt nhất p trong Open (xóa p khỏi Open).
- ▪ Nếu p là trạng thái kết thúc thì thoát.
- ▪ Chuyển p qua Close và tạo ra các trạng thái kế tiếp q sau p .
- ▪ Nếu q đã có trong Open

- ▪ Nếu $g(q) > g(p) + \text{Cost}(p, q)$
- ▪ $g(q) = g(p) + \text{Cost}(p, q)$
- ▪ $f(q) = g(q) + h(q)$
- ▪ $\text{prev}(q) = p$
(đỉnh cha của q là p)
- Chọn trạng thái (đỉnh) tốt nhất p trong Open (xóa p khỏi Open).

- ▪ Nếu p là trạng thái kết thúc thì thoát.
- ▪ Chuyển p qua Close và tạo ra các trạng thái kế tiếp q sau p .
- ▪ Nếu q đã có trong Open
- ▪ Nếu $g(q) > g(p) + \text{Cost}(p, q)$
- ▪ $g(q) = g(p) + \text{Cost}(p, q)$

- ▪ $f(q) = g(q) + h(q)$
- ▪ $\text{prev}(q) = p$
(đỉnh cha của q là p)
- Chọn trạng thái (đỉnh) tốt nhất p trong Open (xóa p khỏi Open).
- ▪ Nếu p là trạng thái kết thúc thì thoát.
- ▪ Chuyển p qua Close và tạo ra

các trạng thái kế
tiếp q sau p .

- ▪ Nếu q đã có
trong Open
- ▪ Nếu $g(q) >$
 $g(p) + \text{Cost}(p, q)$
- ▪ $g(q) = g(p) +$
 $\text{Cost}(p, q)$
- ▪ $f(q) = g(q) +$
 $h(q)$
- ▪ $\text{prev}(q) = p$
(đỉnh cha của q là
 p)

- Chọn trạng thái (đỉnh) tốt nhất p trong Open (xóa p khỏi Open).
- ▪ Nếu p là trạng thái kết thúc thì thoát.
- ▪ Chuyển p qua Close và tạo ra các trạng thái kế tiếp q sau p .
- ▪ Nếu q đã có trong Open

- ▪ Nếu $g(q) > g(p) + \text{Cost}(p, q)$
- ▪ $g(q) = g(p) + \text{Cost}(p, q)$
- ▪ $f(q) = g(q) + h(q)$
- ▪ $\text{prev}(q) = p$
(đỉnh cha của q là p)
 - Chọn trạng thái(đỉnh) tốt nhất p trong Open (xóa p khỏi Open).
 - Nếu p là trạng thái kết thúc thì thoát.
 - Chuyển p qua Close và tạo ra các trạng thái kế tiếp q sau p .
 - Nếu q đã có trong Open
 - Nếu $g(p) > g(p) + \text{Cost}(p, q)$
 - $g(q) = g(p) + \text{Cost}(p, q)$
 - $f(q) = g(q) + h(q)$
 - $\text{prev}(q) = p$ (đỉnh cha của q là p)
 - Nếu q chưa có trong Open
 - $g(q) = g(p) + \text{cost}(p, q)$
 - $f(q) = g(q) + h(q)$
 - $\text{prev}(q) = p$
 - Thêm q vào Open
 - Nếu q có trong Close

- o Nếu $g(q) > g(p) + \text{Cost}(p,q)$
 - Bỏ q khỏi Close
 - Thêm q vào Open
- Bước 3 : Kết thúc vòng lặp

III. Cài đặt thuật toán A* cho bài toán 8-puzzle

Trong việc giải quyết bài toán 8-puzzle, có thể áp dụng nhiều phương pháp tìm kiếm khác nhau như tìm kiếm theo chiều rộng (BFS), tìm kiếm theo chiều sâu (DFS) hoặc tìm kiếm có thông tin như A*. Tuy nhiên, BFS và DFS đều là các thuật toán tìm kiếm mù (blind search), không tận dụng được thông tin về trạng thái đích. Do đó, để đi từ trạng thái ban đầu đến trạng thái đích, hai thuật toán này sẽ phải duyệt qua rất nhiều trạng thái trung gian, tốn nhiều thời gian và số bước di chuyển.

Trong khi đó, A* là một thuật toán tìm kiếm có thông tin (informed search) kết hợp giữa chi phí thực tế đã đi và chi phí ước lượng còn lại. Điều này giúp quá trình tìm kiếm hiệu quả hơn, rút ngắn số lần mở rộng nút và dẫn đến lời giải tối ưu.

Các bước tiến hành cài đặt A* cho bài toán 8-puzzle:

1. Xác định đầu vào, đầu ra và điều kiện dừng
 - o Đầu vào: trạng thái xuất phát và trạng thái đích.
 - o Đầu ra: chuỗi các bước di chuyển để đưa trạng thái ban đầu về trạng thái đích.
 - o Điều kiện dừng: khi thuật toán tìm thấy trạng thái đích.
2. Khởi tạo cấu trúc dữ liệu
 - o Open list: chứa các trạng thái đã sinh ra nhưng chưa mở rộng.
 - o Closed list: chứa các trạng thái đã được mở rộng.
 - o Mỗi trạng thái lưu trữ thông tin $g(n)$, $h(n)$, $f(n)$ cùng con trỏ về trạng thái cha để truy vết lời giải.
3. Chọn hàm heuristic
 - o Số ô sai vị trí (Misplaced tiles): đơn giản, nhanh, nhưng độ chính xác thấp.

- Khoảng cách Manhattan: tính tổng khoảng cách theo trục ngang và dọc của mỗi ô đến vị trí đích. Mặc dù tính toán phức tạp hơn nhưng cho kết quả chính xác hơn, giúp A* tìm kiếm với ít bước hơn.

4. Tính toán hàm đánh giá

$$f(n) = g(n) + h(n)$$

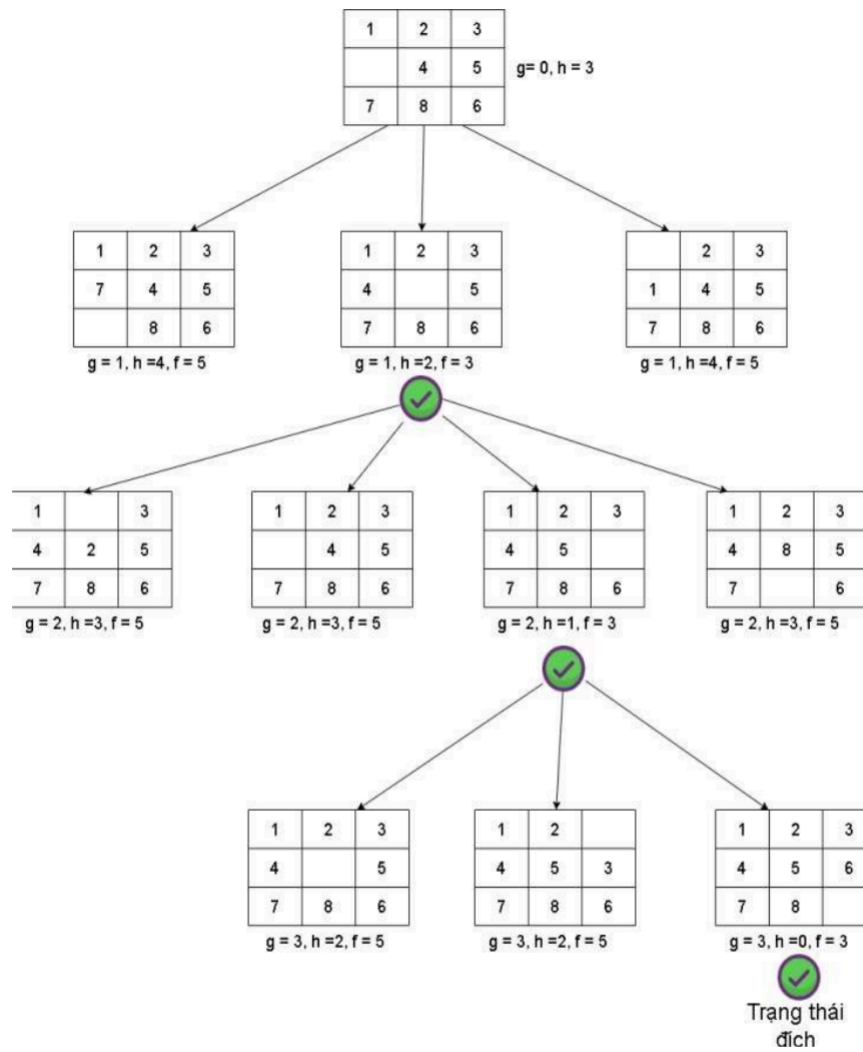
Trong đó:

- $g(n)$: chi phí thực tế từ trạng thái ban đầu đến trạng thái hiện tại.
- $h(n)$: chi phí ước lượng từ trạng thái hiện tại đến trạng thái đích.

5. Thực hiện thuật toán A*

- Bước 1: Khởi tạo open list với trạng thái ban đầu, closed list rỗng.
- Bước 2: Lặp lại cho đến khi open list rỗng:
 - Lấy trạng thái có $f(n)$ nhỏ nhất từ open list → đây là trạng thái hiện tại.
 - Nếu trạng thái hiện tại trùng với trạng thái đích → dừng thuật toán, truy vết đường đi.
 - Nếu chưa phải trạng thái đích: đưa trạng thái hiện tại vào closed list, sinh ra các trạng thái con (di chuyển ô trống lên, xuống, trái, phải nếu hợp lệ).
 - Với mỗi trạng thái con:
 - Tính $g(n), h(n)$
 - Nếu con đã nằm trong closed list → bỏ qua.
 - Nếu con đã nằm trong open list → so sánh và giữ lại trạng thái có $f(n)$ nhỏ hơn.
 - Nếu con chưa có trong open list và closed list → thêm vào open list.
- Bước 3: Nếu open list rỗng mà chưa tìm thấy trạng thái đích → kết luận không tồn tại đường đi.

Minh họa tính toán heuristic:



Giả sử có trạng thái hiện tại:

2	8	3
1	6	4
7		5

và trạng thái đích:

1	2	3
4	5	6
7	8	

- Heuristic 1 – số ô sai vị trí: các ô sai vị trí gồm 2, 8, 1, 6, 4, 5 → tổng cộng 6 ô.
Vậy $h(n)=6h(n) = 6h(n)=6$.
 - Đối với khoảng cách Mahattan, đây là một hàm heuristic chính xác hơn, tính toán tổng khoảng cách theo trục ngang và trục dọc của mỗi ô từ vị trí hiện tại đến vị trí đích của nó. Cách tính như sau: tính khoảng cách Mahatan cho từng ô bằng cách lấy tổng số bước di chuyển theo hàng và cột để đưa ô số từ vị trí hiện tại đến vị trí đích.
- Heuristic 2 – khoảng cách Manhattan:

- o Ô 2: từ (0,0) $\rightarrow$ (0,1) = 1 bước
- o Ô 8: từ (0,1) $\rightarrow$ (2,1) = 2 bước
- o Ô 3: đúng vị trí = 0 bước
- o Ô 1: từ (1,0) $\rightarrow$ (0,0) = 1 bước
- o Ô 6: từ (1,1) $\rightarrow$ (1,2) = 1 bước
- o Ô 4: từ (1,2) $\rightarrow$ (1,1) = 1 bước
- o Ô 7: đúng vị trí = 0 bước
- o Ô 5: từ (2,2) $\rightarrow$ (1,2) = 1 bước

Tổng khoảng cách Manhattan: $1 + 2 + 0 + 1 + 1 + 1 + 0 + 1 = 7$

Kết luận: heuristic “số ô sai vị trí” đơn giản nhưng kém chính xác. Heuristic “khoảng cách Manhattan” tuy tốn công tính toán hơn nhưng ước lượng sát hơn, giúp A* mở rộng ít trạng thái hơn và đạt được lời giải tối ưu

IV. Mã nguồn

1. Khai báo ban đầu

```
import heapq

# Trạng thái đích
TRANG_THAI_DICH = [[1, 2, 3],
                   [4, 5, 6],
                   [7, 8, 0]]
```

-Sử dụng heapq để cài đặt hàng đợi ưu tiên (priority queue), cần thiết cho thuật toán A* (chọn trạng thái có $f = g + h$ nhỏ nhất để mở rộng).

-trang_thai_dich: ma trận 3x3 biểu diễn trạng thái đích.

- +) Số 0 đại diện cho ô trống.
- +) Các số từ 1 đến 8 là các ô gạch.

2. Hàm tiện ích

2.1. Tìm vị trí một số trong ma trận

```
# Tìm vị trí (hàng, cột) của một số trong ma trận
def tim_vi_tri(trang_thai, gia_tri):
    for i in range(3):
        for j in range(3):
            if trang_thai[i][j] == gia_tri:
                return i, j
    return None
```

- Input: trang_thai (ma trận 3x3), gia_tri (giá trị cần tìm).
 - Tìm vị trí (i, j) của một số gia_tri trong ma trận trang_thai.
 - Dùng để tìm vị trí của ô trống (0) hoặc tìm vị trí số trong trạng thái đích.
- 2.2. Hàm heuristic: khoảng cách Manhattan

```
# Hàm heuristic: khoảng cách Manhattan
def khoang_cach_manhattan(trang_thai):
    tong = 0
    for i in range(3):
        for j in range(3):
            gia_tri = trang_thai[i][j]
            if gia_tri != 0:
                dich_i, dich_j = tim_vi_tri(TRANG_THAI_DICH, gia_tri)
                tong += abs(i - dich_i) + abs(j - dich_j)
    return tong
```

- Đây là hàm ước lượng $h(x)$ trong A*.
- Tính tổng khoảng cách Manhattan của từng ô so với vị trí đích.
- Công thức: $h = \sum (|i(\text{hiện tại}) - i(\text{đích})| + |j(\text{hiện tại}) - j(\text{đích})|)$
- Bỏ qua ô trống ($gia_tri \neq 0$).
- Ý nghĩa: tổng số bước các ô cần di chuyển để về đúng vị trí.

2.3. Chuyển trạng thái sang tuple

```
# Chuyển ma trận sang tuple (để lưu trong set)
def ma_tran_sang_tuple(trang_thai):
    return tuple(tuple(hang) for hang in trang_thai)
```


- Chuyển ma trận dạng list sang tuple bất biến để lưu trong set.
- Tránh trùng lặp trạng thái đã duyệt.

2.4. Sinh các trạng thái con (láng giềng)

```
# Sinh các trạng thái kề (di chuyển ô trống)
def trang_thai_ke(trang_thai):
    ket_qua = []
    x, y = tim_vi_tri(trang_thai, 0) # vị trí ô trống
    buoc_di = [(-1,0,"Lên"), (1,0,"Xuống"), (0,-1,"Trái"), (0,1,"Phải")]
    for dx, dy, huong in buoc_di:
        nx, ny = x + dx, y + dy
        if 0 <= nx < 3 and 0 <= ny < 3:
            moi = [hang[:] for hang in trang_thai]
            moi[x][y], moi[nx][ny] = moi[nx][ny], moi[x][y]
            ket_qua.append((moi, huong))
    return ket_qua
```

- Tìm các nước đi hợp lệ từ trạng thái hiện tại.
- Ô trống (x, y) có thể di chuyển lên, xuống, trái, phải (nếu còn trong bảng).
- Sinh ra trạng thái mới sau khi đổi chỗ ô trống.
- Output: danh sách (trạng_thai_mới, hướng_di_chuyển).

3. Thuật toán A*

```
def a_sao(trang_thai_bd):
    hang_doi = []
    heapq.heappush(hang_doi, (khoang_cach_manhattan(trang_thai_bd), 0, trang_thai_bd, []))
    da_tham = set()
```

- hangdoi: hàng đợi ưu tiên (ưu tiên theo $f = g + h$).
- Phần tử trong heap: (f, g, trạng_thái, đường_đi).
- da_tham: tập trạng thái đã xét.

Vòng lặp chính

```

while hang_cho:
    f, g, trang_thai, duong_di = heapq.heappop(hang_cho)
    h = khoang_cach_manhattan(trang_thai)

    # In ra giá trị g, h, f và trạng thái hiện tại
    print("g =", g, ", h =", h, ", f =", f)
    for hang in trang_thai:
        print(hang)
    print("-"*20)

```

-Lấy trạng thái tốt nhất (f nhỏ nhất) ra từ hàng đợi.

-Tính lại h.

-In g, h, f và trạng thái để minh họa.

Kiểm tra trạng thái đích

```

if trang_thai == TRANG_THAI_DICH:
    print("✅ Đã tìm thấy trạng thái đích!")
    return

if doi_sang_tuple(trang_thai) in da_xet:
    continue
da_xet.add(doi_sang_tuple(trang_thai))

```

-Nếu đạt trạng thái đích → dừng lại.

-if doi_sang_tuple(trang_thai) in da_xet:

→ Kiểm tra xem trạng thái hiện tại đã được duyệt (có trong tập da_xet) chưa.

→ Nếu có rồi thì continue (bỏ qua, không xử lý nữa).

→ Nếu chưa có thì thêm trạng thái này vào tập da_xet để đánh dấu là đã duyệt, tránh lặp lại.

Sinh trạng thái con

```

for ke, huong in hang_xom(trang_thai):
    g_moi = g + 1
    h_moi = khoang_cach_manhattan(ke)
    f_moi = g_moi + h_moi
    heapq.heappush(hang_cho, (f_moi, g_moi, ke, duong_di + [(trang_thai, huong)]))

```

-Hàm hang_xom(trang_thai) sinh ra tất cả trạng thái con từ trạng thái hiện tại.

-ke: trạng thái con (sau khi di chuyển ô trống).

-huong: tên bước di chuyển (Lên, Xuống, Trái, Phải).

-g = số bước đã đi từ trạng thái ban đầu đến trạng thái hiện tại.

-Khi sinh thêm một bước đi $\rightarrow g_moi = g + 1$.

-Dùng heuristic Manhattan để ước lượng khoảng cách còn lại từ trạng thái con ke đến trạng thái đích.

- $f(n)$ = tổng chi phí thực tế (g_moi) + ước lượng còn lại (h_moi).

4. Chạy thử

```

trang_thai_ban_dau = [[1, 2, 3],
                      [4, 5, 0],
                      [7, 8, 6]]

loi_giai = a_sao(trang_thai_ban_dau)

if loi_giai:
    for i, (s, buoc) in enumerate(loi_giai):
        print(f"Bước {i}: Di chuyển {buoc}")
        for hang in s:
            print(hang)
        print()
else:
    print("Không tìm được lời giải")

```

-Đặt trạng thái ban đầu.

-Gọi hàm A* để tìm lời giải.

-Hàm trả về loi_giai là danh sách các bước (nếu tìm được), thường có dạng:
[(state0, move0), (state1, move1), ..., (stateN, "Đích")], mỗi phần tử chứa state (ma trận)
và move (hướng di chuyển).

- Kiểm tra xem loi_giai có giá trị hay không.

-Trong Python, None hoặc [] (danh sách rỗng) được đánh giá là False; nếu hàm trả về
danh sách không rỗng thì True.

-Duyệt từng phần tử trong loi_giai bằng enumerate để có số bước i (bắt đầu từ 0) và nội
dung (s, buoc):

+) s là trạng thái (ma trận) tại bước đó.

+)buoc là hướng di chuyển đã thực hiện để đến s.

-In ra dòng thông báo: Bước i: Di chuyển <hướng>.

-Bên trong mỗi bước: for hang in s in từng hàng của ma trận s để hiển thị bảng 3×3 theo
dạng dòng.

-Nếu loi_giai là None hoặc rỗng → in thông báo rằng không có lời giải (trạng thái vô
nghịệm hoặc hàm không tìm được đường đi).

5. Kết quả

```
⇒ g = 0 , h = 1 , f = 1
   [1, 2, 3]
   [4, 5, 0]
   [7, 8, 6]
   -----
   g = 1 , h = 0 , f = 1
   [1, 2, 3]
   [4, 5, 6]
   [7, 8, 0]
```

```
        if tmp != None:
            S = tmp
    return S,G
```

- Chạy thuật toán:

```
t1 = time.time()
S, G = init(10)
RUN()
t2 = time.time()
print('time: ', t2 - t1)
```

- Khởi tạo trạng thái ban đầu S và trạng thái đích G.

- Chạy thuật toán A^* để tìm đường đi từ S đến G.

- # Tính toán và in ra thời gian thực thi.

Bài toán 8-puzzle là một thách thức kinh điển trong lĩnh vực trí tuệ nhân tạo, đòi hỏi sự áp dụng hiệu quả của các thuật toán tìm kiếm. Trong báo cáo này, chúng em đã trình bày chi tiết về cách sử dụng thuật toán A* để giải quyết bài toán 8-puzzle, từ việc mô tả thuật toán, cài đặt đến việc đánh giá các hàm heuristic khác nhau. Thuật toán A* đã chứng minh được hiệu quả vượt trội trong việc tìm kiếm đường đi ngắn nhất từ trạng thái ban đầu đến trạng thái đích trong bài toán 8-puzzle. Điều này đạt được nhờ vào việc kết hợp chi phí thực tế từ điểm khởi đầu đến điểm hiện tại ($g(n)$) và chi phí ước lượng từ điểm hiện tại đến điểm đích ($h(n)$). Chúng em đã thử nghiệm và so sánh hai hàm heuristic phổ biến là số ô không đúng vị trí và khoảng cách Manhattan. Kết quả cho thấy, khoảng cách Manhattan tuy phức tạp hơn nhưng mang lại độ chính xác cao hơn, dẫn đến việc giảm thiểu số bước tìm kiếm cần thiết.

Qua quá trình nghiên cứu và thực hiện đề tài, chúng em đã rút ra được những bài học quý báu về cách áp dụng thuật toán A* trong việc giải quyết các bài toán tìm kiếm. Hơn nữa, việc hiểu rõ và lựa chọn hàm heuristic phù hợp có vai trò quan trọng trong việc tối ưu hóa quá trình tìm kiếm.

Kết quả của đề tài không chỉ dừng lại ở việc giải quyết bài toán 8-puzzle mà còn mở ra những hướng nghiên cứu và ứng dụng rộng hơn trong các lĩnh vực khác như lập kế hoạch di chuyển cho robot, phát triển các trò chơi giải đố, và tối ưu hóa đường đi trong hệ thống tự động hóa.

Lời cảm ơn

Cuối cùng, chúng em xin chân thành cảm ơn sự hướng dẫn tận tình của thầy Phạm Hồng Phong và sự hỗ trợ từ các tài liệu tham khảo đã giúp chúng em hoàn thành báo cáo này. Hy vọng rằng, những kết quả và kinh nghiệm thu được từ đề tài này sẽ đóng góp vào sự phát triển của lĩnh vực trí tuệ nhân tạo và các ứng dụng thực tiễn trong tương lai.

VI. Tài liệu tham khảo

[1] A* Search Algorithm:

<https://www.geeksforgeeks.org/a-search-algorithm/>

[2] Thuật toán A* algorithm:

[https://www.youtube.com/watch?](https://www.youtube.com/watch?v=G7XnNtF7UEE&list=PLQj93CJe0N73raStQrnbZ9oSLtYZv8zej&index=8)

[v=G7XnNtF7UEE&list=PLQj93CJe0N73raStQrnbZ9oSLtYZv8zej&index=8](https://www.youtube.com/watch?v=G7XnNtF7UEE&list=PLQj93CJe0N73raStQrnbZ9oSLtYZv8zej&index=8)

[3] “8 Puzzle Problem in AI – GeeksforGeeks”:

trang web GeeksforGeeks.

[4] “Heuristic Functions”:

trang giáo trình & bài tập của môn Heuristic Search / Artificial Intelligence, ví dụ như cs.utexas.edu và các trường đại học khác.